

Take-home: Inspection Report Form

Build a small inspection form that keeps unsaved work safe and never saves the same report twice. Plan on about 3–4 hours, and please don't spend more than 5.

Send us a link to a GitHub repo that includes a README.

The scenario

A warehouse company sends inspectors to check equipment such as forklifts and fire extinguishers. Inspectors fill in a report on a laptop, and the warehouse Wi-Fi is unreliable.

Inspectors have two complaints:

1. They lose half-filled reports when the browser closes or the connection drops.
2. Some reports get saved twice when they press Submit on a slow connection.

Each equipment type needs different questions, and the operations team changes those questions often. So the form must be built from a definition, not hard-coded.

What to build

1. A form rendered from JSON

Use this definition exactly as given:

```
JSON
{
  "id": "forklift-check",
  "version": 3,
  "title": { "en": "Forklift Inspection", "ar": "فحص الرافعة الشوكية" },
  "fields": [
    { "key": "serial", "type": "text", "required": true,
      "label": { "en": "Serial number", "ar": "الرقم التسلسلي" } },
    { "key": "hours", "type": "number", "required": true, "min": 0, "max": 50000,
      "label": { "en": "Operating hours", "ar": "ساعات التشغيل" } },
    { "key": "condition", "type": "select", "required": true,
      "options": [
        { "value": "ok", "label": { "en": "OK", "ar": "سليم" } },
        { "value": "damaged", "label": { "en": "Damaged", "ar": "تالف" } }
      ],
      "label": { "en": "Condition", "ar": "الحالة" } },
    { "key": "damageNotes", "type": "textarea", "required": true,
      "showIf": { "field": "condition", "equals": "damaged" },
      "label": { "en": "Describe the damage", "ar": "صف التلف" } },
    { "key": "inspectedOn", "type": "date", "required": true,
      "label": { "en": "Inspection date", "ar": "تاريخ الفحص" } }
  ]
}
```

- Support at least these field types: `text`, `number`, `select`, `textarea` and `date`.
- Show clear validation messages.
- A field hidden by `showIf` must not block submission and must not be sent.

2. Drafts that survive

- If the user refreshes, closes the tab or loses the connection, the form comes back exactly as they left it.

- Clear the draft only after the server confirms the report was saved.

3. Safe submit

- Pressing Submit several times, or retrying after a timeout, must never create two reports.
- The user should always know the state: saving, saved, or failed with a way to retry.

4. English and Arabic

- A language toggle that also switches the layout to right-to-left.

5. A small mock server

Use Node with Express, or anything similar.

- `POST /reports` saves the report in memory.
- It waits a random 0–3 seconds before responding.
- It fails about 30% of the time. Some failures must happen after the report is saved, so the client never gets the response even though the save went through.
- `GET /reports` lists the saved reports, so we can check for duplicates.

Tech and README

React and TypeScript are required. You choose everything else: libraries, styling and storage.

Your README should cover:

- How to run the project, in one or two commands.
- In a few sentences, how you handled drafts and safe submit.
- What you would do with more time.
- Which AI tools you used, if any, and what for. Using them is fine, but you must be able to explain every line.

Follow-up call

We'll have a 30-minute call where you walk us through your code. During the call we'll ask you to make one small change live.